Code Explanation

Code Explanation: AWS Glue PySpark Job - Customer Order Analytics with SCD Type 2

Overview

This AWS Glue PySpark job implements a complete ETL pipeline for customer and order data analytics. It ingests CSV data from S3, cleans it, applies Slowly Changing Dimension (SCD) Type 2 tracking using Apache Hudi, and generates analytical outputs.---

Step-by-Step Breakdown

1. **Imports and Logging Setup**

- Imports standard Python libraries (sys, logging, datetime, typing)- Imports AWS SDK (boto3) for S3 operations- Imports PySpark components for distributed data processing- Imports AWS Glue utilities for job management- Configures logging with timestamp, logger name, level, and message format

2. **SparkContextManager Class**

Purpose: Initializes Spark and Glue execution contexts**Key Method: `initialize_spark_contexts()`**- Creates or retrieves existing SparkContext- Wraps it in GlueContext for AWS Glue integration- Extracts SparkSession for DataFrame operations- Configures Spark for performance: - Enables adaptive query execution - Sets partition coalescing - Configures 200 shuffle partitions- Configures Hudi-specific settings: - Kryo serializer for efficient object serialization - Disables Parquet conversion for Hive metastore compatibility

3. **S3Validator Class**

Purpose: Validates S3 paths and access permissions**Method: `validate_s3_path(s3_path)`**- Checks if path starts with "s3://"- Parses bucket and prefix from path- Calls S3 `head_bucket` to verify access permissions- Returns True if accessible, False otherwise**Method: `check_s3_path_exists(s3_path)`**- Lists objects at the S3 path (max 1 object)- Returns True if any objects exist at the location- Used to verify data availability before reading

4. **DataReader Class**

Purpose: Safely reads data from S3 with validation**Method: `read_data_safe(s3_path, file_format, schema, **options)`**- Validates S3 path accessibility- Checks if path contains data- Configures Spark reader with format and optional schema- Applies additional read options (header, delimiter, encoding)- Normalizes all column names to lowercase for consistency- Logs record count after successful read- Returns DataFrame or None on failure

5. **DataWriter Class**

Purpose: Safely writes data to S3 with validation**Method: `write_data_safe(df, s3_path, file_format, mode, **options)`**- Validates S3 destination path- Configures Spark writer with format and write mode- Applies additional write options- Executes write operation- Logs record count after successful write- Returns True on success, False on failure

6. **DataCleaner Class**

Purpose: Implements data quality rules**Method: `clean_data(df, null_string_values)`**- **Step 1:** Removes rows with actual NULL values using `df.na.drop()`- **Step 2:** Identifies string representations of NULL ('Null', 'NULL', 'null', 'None', etc.) - Replaces these strings with actual NULL values - Drops rows containing these converted NULLs- **Step 3:** Removes duplicate records using `dropDuplicates()`- Logs counts after each cleaning step- Returns cleaned DataFrame

7. **SCDType2Handler Class**

Purpose: Implements Slowly Changing Dimension Type 2 logic using Apache Hudi**Method: `add_scd2_columns(df, is_initial_load)`**- Adds four tracking columns: - `isactive` (Boolean): True for current records - `startdate` (Timestamp): When record became effective - `enddate` (Timestamp): When record expired (NULL for active) - `opts` (Timestamp): Operation timestamp for versioning- Sets all records as active with current timestamp on initial load**Method: `write_hudi_table(df, s3_path, table_name, record_key, precombine_field, operation)`**- Configures Hudi options: - Table name for identification - Record key (primary key) for deduplication - Precombine field (timestamp) for conflict resolution - Operation type (upsert merges updates/inserts) - COPY_ON_WRITE table type for read optimization - Parallelism settings for performance- Writes DataFrame in Hudi format to S3- Returns success status**Method: `read_hudi_table(s3_path)`**- Reads Hudi table from S3- Normalizes column names to lowercase- Returns DataFrame or None on failure

8. **CustomerOrderAnalytics Class**

Purpose: Main orchestration class implementing all functional requirements**Initialization:**- Stores Spark and Glue contexts- Instantiates helper classes (validator, reader, writer, cleaner, SCD2 handler)- Defines S3 paths for inputs and outputs**Schema Methods:**- `get_customer_schema()`: Defines customer table structure (CustId, Name, EmailId, Region)- `get_order_schema()`: Defines order table structure (OrderId, ItemName, PricePerUnit, Qty, Date)**Data Ingestion Methods:****`ingest_customer_data()`** (FR-INGEST-001)- Reads customer CSV from S3- Applies customer schema- Configures CSV options (header, UTF-8 encoding, comma delimiter)- Returns customer DataFrame**`ingest_order_data()`** (FR-INGEST-002)- Reads order CSV from S3- Applies order schema- Configures CSV options- Returns order DataFrame**Data Cleaning Methods:****`clean_customer_data(df)`** (FR-CLEAN-001)- Applies data cleaning rules to customer data- Removes NULLs, 'Null' strings, and duplicates- Returns cleaned DataFrame**`clean_order_data(df)`** (FR-CLEAN-001)- Applies data cleaning rules to order data- Same cleaning logic as customer data- Returns cleaned DataFrame**SCD Type 2 Method:****`implement_scd2_customer(df)`** (FR-SCD2-001)- Adds SCD Type 2 tracking columns to customer data- Writes to Hudi table with upsert operation- Uses `custid` as record key and `opts` as precombine field- Enables historical change tracking- Returns success status**Analytics Methods:****`calculate_customer_aggregate_spend(customer_df, order_df)`** (FR-AGG-001)- Calculates total amount per order: `PricePerUnit * Qty`- Joins customer and order data on `custid`- Groups by customer ID and name- Sums total spend per customer- Returns aggregate DataFrame**`generate_order_summary(customer_df, order_df)`** (FR-SUMMARY-001)- Calculates total amount per order- Joins customer and order data- Selects summary columns: OrderId, CustId, Name, ItemName, TotalAmount, Date- Returns summary DataFrame**Output Methods:****`write_customer_aggregate_spend(df)`**- Writes aggregate spend to S3 in Parquet format- Uses overwrite mode- Returns success status**`write_order_summary(df)`**- Writes order summary to S3 in Parquet format- Uses overwrite mode- Returns success status**Pipeline Orchestration:****`run()`** - Executes complete ETL pipeline:1. Ingests customer data2. Ingests order data3. Cleans customer data4. Cleans order data5. Implements SCD Type 2 for customer data6. Calculates customer aggregate spend7. Generates order summary8. Writes aggregate spend to S39.

Writes order summary to S3- Returns True if all steps succeed, False otherwise- Logs progress and errors at each step

9. **Main Entry Point**

`main()` function:- Retrieves job parameters (JOB_NAME) from command-line arguments- Initializes Spark and Glue contexts- Initializes and registers Glue job for tracking- Creates CustomerOrderAnalytics instance- Executes the ETL pipeline- Commits the Glue job- Exits with status code 0 (success) or 1 (failure)**`if __name__ == "__main__":`**- Ensures main() only runs when script is executed directly- Prevents execution when imported as a module---

Data Flow Summary

1. **Ingest** → Read customer and order CSVs from S32. **Clean** → Remove NULLs, 'Null' strings, and duplicates3. **Track** → Apply SCD Type 2 to customer data using Hudi4. **Analyze** → Calculate customer aggregate spend and order summaries5. **Output** → Write results to S3 in Parquet format---

Key Technical Features

- **Error Handling:** Try-catch blocks with detailed logging at every step- **Validation:** S3 path and data existence checks before operations- **Traceability:** Each method maps to specific functional requirements (FR-XXX-XXX)- **Scalability:** Spark distributed processing with configurable parallelism- **Data Quality:** Multi-step cleaning process with audit logging- **Historical Tracking:** SCD Type 2 implementation preserves data lineage- **Performance:** Adaptive query execution and optimized Hudi configurations